ADR 0015 — A closed Git-operation vocabulary and the reviewable Plan schema

- **Status:** Accepted
- **Date:** 2026-07-24
- **Milestone / issue:** M1.06a — Define the typed operation vocabulary and plan schema (#142, child of #59)
- **Supersedes / superseded by:** — (builds on ADR 0001's generation and ADR 0003's opaque-id catalog; execution-time enforcement is #145)

Context

The Foundation exit criterion (M1.06, #59) requires that every Git mutation be previewed as a reviewable plan before it runs, and that a stale browser tab can never execute against a state the user didn't review. Today the write handlers each accept a small ad-hoc DTO (`BranchRequest`, `CreateCommitRequest`, …) and run git immediately; there is no shared vocabulary naming what kinds of mutation exist, and no shape a preview, approval, or execution-time check could be built on.

An audit of every write route in `git-vista-server` (branch.rs, commit.rs, rebase.rs, reset.rs, clone.rs, activity.rs's undo) found exactly fifteen distinct mutations of the served repository in use, and four write endpoints that mutate no served repository at all (`/api/clone`, `/api/delete-clone`, `/api/select`, `/api/rescan` — they manage the catalog / app session).

Decision

A closed vocabulary: `GitOperation` in `git-vista-protocol`

`git_vista_protocol::plan::GitOperation` is an internally-tagged (`"op"`, `snake_case`) enum with **one variant per audited mutation and no catch-all**: `create_branch`, `commit_on_head`, `empty_commit_on_branch`, `stage_all`, `unstage_all`, `checkout_branch`, `merge_branch`, `push_branch`, `delete_branch`, `force_delete_branch`, `rebase_onto_base`, `restore_branch`, `reset_branch`, `revert_commit`, `reset_test_repo`. An unknown tag fails to deserialize — a new kind of mutation must extend the enum, visibly. The two `/api/commit` code paths are deliberately two variants: a plain `git commit` on HEAD and the commit-tree + compare-and-swap `update-ref` written onto a branch that isn't checked out are different mutations, with different preconditions and different ref effects.

Scope: served-repository mutations only. `/api/clone`, `/api/delete-clone`, `/api/select` and `/api/rescan` are not operations. They never move a served repository's refs, index, or working tree — the state ADR 0001's generation is computed over and a plan's preconditions are defined against. Clone has no target worktree or generation before it runs; select/rescan change only which repository the app serves. Folding them in would have forced every plan field to become optional, hollowing out exactly the mechanical checks #145 needs. They remain guarded by their existing session/route machinery (ADRs 0004/0005).

The `Plan` schema

`Plan` is the reviewable preview of one operation — everything a user approves before the mutation runs:

- `repository` / `worktree` — opaque id tokens (the string forms of ADR 0001's ids, addressed through ADR 0003's catalog; never a path).
- `generation` — the reviewed state, as an **opaque string token** compared only for equality, exactly as ADR 0001's versioning note prescribes (today the decimal form of the core `u64`; a future algorithm version can add a discriminator without a wire break).
- `operation` — the `GitOperation` variant itself.
- `operation_hash` — SHA-256 (64 lowercase hex) of the operation's canonical JSON (the exact `serde_json` bytes; struct field order is fixed), binding an approval to one operation.
- `issued_at` / `expires_at` — Unix seconds; an expired plan is refused.
- `risk` — `safe` / `reversible` / `destructive` / `remote`, so the UI can scale its confirmation ceremony to what approval risks.
- `preconditions` — a typed list the server re-checks live at execution time (`ref_at` compare-and-swap, `ref_exists` / `ref_absent`, `branch_checked_out` / `branch_not_checked_out`, `clean_worktree`, `remote_configured`, `seed_recorded`).
- `expected_ref_changes` — per-ref `before`/`after` states, where a state is `absent`, `at` an exact oid, `symbolic` (HEAD across a checkout), or `computed` (produced by the operation, unknowable until it runs).
- `recovery` — a typed strategy (`not_needed`, `reset_ref`, `recreate_branch`, `delete_created_branch`, `checkout_previous`, `revert_commit`, `irrecoverable`), so the UI can say how to get back, and a later milestone can offer it.

Nothing is free text. Every string-shaped field is a validating newtype whose `Deserialize` rejects malformed input at the wire (empty names, option-shaped `-`-prefixed values, non-hex ids) — the same belt-and-braces gates the write handlers apply today, moved to the type boundary. `Plan` itself carries `deny_unknown_fields`, like every request body since ADR 0002.

Where it lives, and the serialization contract

The types live in a new `plan` module of `git-vista-protocol`: they cross the HTTP/JSON wire, so per ADR 0002 they belong to the transport crate — pure, wasm-safe, serde-only — shared verbatim by the server and the Leptos frontend. The protocol crate still does not depend on `git-vista-core`; ids and the generation cross as opaque tokens, mirroring `RepositoryDescriptor`.

The wire form is pinned by a **committed golden fixture** (`crates/git-vista-protocol/tests/fixtures/plan_v1.json`): fifteen plans, one per operation variant, exercising every risk level, precondition, ref-state and recovery variant. The test proves losslessness both ways — the fixture deserializes into exactly the in-code plans, and re-serializing reproduces the committed bytes — so any rename or retag is a loud, deliberate change (regenerated via `REGEN_GOLDEN=1`, reviewed as a diff). No endpoint serves a `Plan` yet, so `PROTOCOL_VERSION` stays at 2; the first endpoint to carry one follows the M1.02 rules.

Division of labour with #145

This ADR defines the shapes; #145 makes them load-bearing at execution time: recompute the operation hash and refuse a mismatch, compare the live worktree generation for equality, refuse past `expires_at`, and evaluate each precondition against the live repository. All four checks are plain typed comparisons over fields defined here — that is the design's success criterion.

Alternatives considered

- **A generic `{ command, args }` escape hatch.** Rejected outright: it reintroduces stringly git at the exact seam the milestone exists to close, and makes preview/undo/risk analysis impossible. The closed enum is the point.
- **One `Commit` variant covering both `/api/commit` paths.** Rejected: the HEAD path and the stub-branch path differ in mechanics (index vs. commit-tree), preconditions (checked-out vs. not), and ref effects; merging them would need optional fields and conditional meaning — stringly typing with extra steps.
- **Including clone/select/rescan/delete-clone in the vocabulary.** Rejected for the scope reasons above; recorded here so the exclusion is a decision, not an oversight.
- **Defining the types in `git-vista-core` and re-exporting.** Rejected: ADR 0002 separates transport from domain precisely so the wire contract versions independently; core stays free of transport concerns and the protocol crate stays dependency-free (serde only).
- **A structural `{ verbs × targets }` grammar instead of an enum of whole operations.** More "algebraic", but it admits combinations no handler implements (e.g. force-delete a tag) — the vocabulary would no longer be exactly what the app can do, and the golden fixture could not enumerate it.

Consequences

- There is now one place that answers "what can git-vista do to a repository?" — fifteen variants, each mapped in the module docs to the endpoint and git invocation it describes. A sixteenth mutation cannot be added quietly: it fails the vocabulary-coverage test until it has a golden plan.
- **145 (execution-time enforcement) and the M9 preview/rehearsal work build on** these shapes without re-deciding them; the handlers' ad-hoc DTOs (`BranchRequest` etc.) keep serving the current endpoints until the plan pipeline replaces them.
- The operation-hash canonical form is fixed as "the operation's own `serde_json` bytes", so both sides can compute it with what they already ship; if a canonicalisation subtlety ever surfaces (e.g. float-free JSON is already guaranteed here), it is a superseding ADR.
- Fixture regeneration is a reviewed, explicit act; CI (the `./dev gate` test step) fails on any accidental wire drift.